



Kim

AI-assisted parametric design for functional 3D prints

Full Technical Reference

Version:	0.9.2 Windows beta
Platform:	Windows 11 / macOS / Linux (from source)
Source:	github.com/scottconverse/KimCadClaude
License:	MIT

This document covers the full product - the user-facing interface, the technical surface for developers and integrators, and the internal architecture. For a quick start, see the README at the repository root.

Contents

Part 1 · Everyday Use

- What KimCad is (and isn't)
- Installing on Windows
- Installing on macOS / Linux
- First run & the setup wizard
- The three ways to start a design
- Live parameter sliders
- Refining a design
- Printing - confirming and slicing
- Sending directly to a printer
- My Designs library
- What to do when things go wrong
- Glossary

Part 2 · The Technical Surface

- CLI reference
- Configuration
- Printer connectors
- The MCP server
- The CadQuery / STEP engine
- API reference

Part 3 · Architecture

- The pipeline - diagram
- Module map
- The template engine & catalog
- The web layer
- Local-first design & the injectable seam
- Trust boundaries
- Testing philosophy

Part 1

Everyday Use

What KimCad is (and isn't)

KimCad turns a description of a functional part - a bracket, a holder, a clip, an enclosure - into a 3D-printable file. You describe it in plain words, or start from a photo or a rough sketch. KimCad designs it, checks it's actually printable, and gives you a ready-to-slice file. You never draw anything or edit any code.

It runs entirely on your own computer. No account is needed, no internet is required, and nothing you type, photograph, or sketch ever leaves your machine - unless you explicitly turn on an optional cloud AI feature in Settings.

KimCad is best at single mechanical parts. It is not a freeform artistic modeler and not a multi-part assembly tool.

Installing on Windows

The easiest path is the double-click installer - no terminal, no Python, no developer tools needed.

- * Download KimCad-Setup-0.9.2.exe from the Releases page on GitHub.
- * Double-click it. Windows will warn you because the beta isn't code-signed - click More info -> Run anyway. (The release page publishes a checksum you can verify first.)
- * Follow the wizard. KimCad installs to Program Files by default.
- * Launch from the Start Menu shortcut.

Requirements: Windows 11 (or Windows 10 with the WebView2 Runtime, which Edge installs automatically), about 12 GB free disk space for the AI and its models, 16 GB+ RAM recommended. No graphics card needed.

Installing on macOS and Linux

The zero-terminal installer is Windows-only for now. On macOS and Linux you run KimCad from the source code.

```
pip install kimgcad
kimgcad web
```

You install OpenSCAD and OrcaSlicer yourself and point the config file at them. Your saved designs and settings land in the platform-standard location (~/.Library/Application Support/KimCad on macOS, ~/.local/share/KimCad on Linux).

First run and the setup wizard

The first time you launch KimCad, a setup wizard walks you through three things:

- * The AI. KimCad runs its design intelligence locally through Ollama (free, included). It sets Ollama up for you - you don't install anything separately. On first launch it downloads the engine (~1.4 GB) and then the design model (~4.7 GB). This takes a few minutes on a reasonable connection.
- * Your printer. Pick from the catalog of ~29 supported printers, or add your own.
- * Your material. Choose the filament you're loading.

After setup, KimCad stays ready - Ollama runs in the background while the app is open and shuts down cleanly when you close it. Models are only downloaded once.

The three ways to start a design

1. Type a description

The simplest path: type what you want in the text field and press Enter. Be specific about dimensions - 'a cable clip for a 6 mm cable' gets you better results than 'a cable clip.'

2. Start from a photo

Click the camera icon and upload a photo of an existing part. KimCad's vision model reads the photo and turns it into a text description that seeds the design. Review the seed before proceeding - the vision read is a starting point, not a measurement.

3. Start from a sketch

Draw a rough sketch with dimensions written on it, photograph it, and upload it. KimCad reads the sketch and takes the written dimensions as given.

Live parameter sliders

When KimCad uses one of its built-in templates - which covers the vast majority of common shapes - the design comes back with sliders for every parameter: width, height, depth, wall thickness, and more. Drag a slider and the 3D view updates instantly, with no AI call. The geometry is rebuilt from scratch in well under a second.

The slider ranges are bounded to values that will actually print, so you can't accidentally dial in something the printability check would reject.

Refining a design

After a design comes back, you can ask KimCad to adjust it in plain language: 'make the walls thicker', 'add a mounting hole', 'taller'. Each refinement is a new version; you can go back to any previous version from the history panel.

Printing - confirming and slicing

When you're happy with the design, KimCad shows you the Printability Gate result - pass, warning, or fail, with a plain-English explanation. A passing part gets a Slice & prepare button. Click it and KimCad runs the slicer and shows you the estimated print time, layers, and filament use. The sliced file is ready to download or send directly to your printer.

KimCad never slices anything automatically. You always click a button to confirm before the slicer runs, and again before anything is sent to a printer.

Sending directly to a printer

After slicing, a Send panel appears. Pick the printer from the list and KimCad sends the file directly - no SD card, no USB drive. Supported connection types:

- * Bambu Lab (LAN mode) - P2S, A1, and compatible models
- * OctoPrint
- * Moonraker / Klipper (Voron, Ender with Klipper, RatRig, Mainsail, Fluidt)
- * PrusaLink - MK4, MK3.9, MINI, XL
- * Duet / RepRapFirmware
- * Marlin over USB serial or serial-over-network

In this beta, every connector has been tested against a software simulator that faithfully mimics the real printer's protocol. Testing against actual hardware is the beta's main job (tracked in issue #11).

My Designs library

Every design you build is automatically saved to My Designs on your machine. You can name them, browse them in a gallery, open an old design to adjust it, duplicate it as a starting point, or export it as a .kimcad file to move to another machine.

What to do when things go wrong

The AI seems slow	Planning a part takes 30-120 seconds on a typical laptop CPU - this is normal. The timer in the UI shows progress.
'KimCad couldn't reach your local AI'	The AI engine isn't running. Go to Settings -> AI setup and restart it.
Printability check failed	The part has a geometry or size problem. Read the plain-English reason and adjust your sliders or description.
The part is the wrong size	Check that your description included explicit dimensions. Add them and try again.
SmartScreen warning on install	Expected - the beta isn't code-signed yet. Click 'More info -> Run anyway'. You can verify the SHA-256 checksum on the release page first.
Slice button is greyed out	The printability check didn't pass. Fix the geometry first.
Settings panel closes when I click a section	Update to 0.9.2 - this was fixed.

Glossary

Ollama	The free, local AI runtime KimCad uses to run its design intelligence on your machine.
Template engine	KimCad's built-in library of 86 common shapes. When your part matches a known shape, geometry is built without any AI call.
Printability Gate	The check that tells you whether a part will print successfully on your chosen printer with your chosen material.
OrcaSlicer	The free slicing tool KimCad uses to convert a 3D model into the G-code your printer reads.

G-code

The instruction file a 3D printer reads. Every move, temperature, and speed is in this file.

Smart Mesh Card

The readiness summary KimCad shows after a design is built - score, risks, and plain-English recommendations.

STEP / .kimcad

Export formats. STEP is editable in professional CAD tools. .kimcad is KimCad's portable archive format.

MCP server

An optional interface that lets AI assistants (like Claude) drive KimCad programmatically.

Part 2

The Technical Surface

CLI reference

All commands run as `kimcad <verb> [options]`.

kimcad design "<prompt>"	Design a part. Add <code>--slice</code> to also slice it. Add <code>--send <name></code> to send to a named printer.
kimcad web [--port N]	Start the browser UI at localhost:8765 (or custom port).
kimcad shell	Launch the windowed app (Windows). Falls back to kimcad web on macOS/Linux.
kimcad models	Show the hardware-aware AI model recommendation for this machine.
kimcad bakeoff	Run a benchmark across multiple AI models and recommend the best one.
kimcad bench [--slice]	Run the built-in design benchmark suite.
kimcad --version	Print the version.

Configuration

Configuration is layered: `config/default.yaml` (shipped, read-only) is overlaid by `config/local.yaml` (gitignored, per-user). In installed mode, `local.yaml` lives in `%LOCALAPPDATA%\KimCad` on Windows.

Key settings:

backends.local.model	The planning model. Default: <code>qwen2.5:7b</code> .
backends.local.vision_model	The vision model for photos/sketches. Default: <code>qwen2.5vl:3b</code> .
printers	Named printer configurations with build volume, nozzle size, and connection settings.
materials	Named material profiles with density, print temperature, and bed temperature.

Credentials (API keys, Bambu access codes, OctoPrint keys) are always read from environment variables - never stored in config files, never logged.

Printer connectors

Each connector is validated end-to-end against a runnable simulator that faithfully mimics the real printer's protocol, including fault injection.

bambu	Bambu Lab LAN mode. MQTT-over-TLS for control + FTPS upload. Access code from env var <code>BAMBU_ACCESS_CODE</code> .
octoprint	OctoPrint REST API. API key from env var <code>OCTOPRINT_API_KEY</code> .
moonraker	Moonraker (Klipper). Optional API key from env var <code>MOONRAKER_API_KEY</code> .

prusalink

PrusaLink REST API. API key from env var PRUSALINK_API_KEY.

duet

Duet/RepRapFirmware via /rr_* HTTP interface. Optional board password from env var DUET_PASSWORD.

marlin

Marlin via USB serial or serial-over-network. Port from env var MARLIN_PORT.

mock

In-memory loopback connector for development and testing.

The MCP server

KimCad ships a printer MCP (Model Context Protocol) server that lets AI assistants drive the printer connectors programmatically.

```
python -m kimcad.mcp_server
```

Exposed tools:

- * `list_connectors` - list configured connectors with capabilities
- * `printer_status` - get live status of a named connector
- * `printer_capabilities` - get the printer's reported build volume and nozzle
- * `send_print` - send a sliced file to a named printer (requires `confirm=true` - the exact boolean)

The CadQuery / STEP engine

For supported template families, KimCad also builds an editable STEP file alongside the STL - the kind you can open in Fusion 360, FreeCAD, or any professional CAD tool. This uses CadQuery, an optional dependency.

- * CadQuery is entirely optional. If it's not installed, STEP export is silently unavailable - everything else works.
- * KimCad authors every CadQuery script - no AI ever writes CadQuery. The STEP path carries none of the AI-codegen risk.
- * The worker runs in a separate process with a restricted environment as defense-in-depth.

API reference (web layer)

All endpoints are on localhost only (127.0.0.1) by default.

GET /api/health	Liveness check. Returns version and status.
POST /api/design	Run a design. Body: {prompt, printer, material}. Returns plan, gate, mesh_url, parameters.
POST /api/render/<id>	Re-render with new parameter values. Body: {values}. No AI call.
POST /api/slice/<id>	Slice the validated mesh. Returns time/layers/filament estimate.
GET /api/gcode/<id>	Download the sliced 3MF.
POST /api/send/<id>	Send to printer. Body: {connector, confirm: true}.
GET /api/step/<id>	Download the STEP/BREP file (CadQuery-backed designs only).
POST /api/photo-seed	Upload a photo. Returns a text seed for a design prompt.
POST /api/sketch-seed	Upload a sketch. Returns a text seed.
GET /api/designs	List saved designs.
POST /api/designs/save	Save the current design with a thumbnail.
GET /api/model-status	Report which AI models are installed and ready.
POST /api/model-pull	Start downloading KimCad's own AI models.

State-changing POSTs require an X-KimCad-Session header containing the per-boot session token injected into the

page. This protects against cross-origin requests from malicious pages. GET requests are never gated.

Part 3 · Architecture

The Design Pipeline

Every design request flows through a deterministic pipeline. The AI model participates in exactly two steps - writing the structured design plan and, only when no template matches, generating OpenSCAD geometry. Every other stage is ordinary, testable code.



The AI is injected at two points only. Every other stage is deterministic - the renderer, gate, orientation, hardening, and slicer are all ordinary code with no model calls. This makes the pipeline fully testable offline and means a geometry mistake is fixed by changing parameters, not by re-running the model.

Part 3 · Architecture

Module Map

Every Python module in `src/kimcad/` has a single, bounded responsibility. The table below gives the purpose of each.

pipeline.py	The orchestrator. Wires every stage, owns the render/gate retry loop, enforces confirm-before-slice.
ir.py	The Design-Plan IR (Pydantic). Validates AI JSON before any geometry is written.
llm_provider.py	All AI communication - planning, OpenSCAD codegen, photo/sketch vision reads.
templates.py	The deterministic template engine. 86 parametric families, pure string substitution.
openscad_runner.py	Sanitize-and-render. Blocks dangerous OpenSCAD constructs before the binary runs.
cadquery_runner.py	STEP export via CadQuery. Runs trusted template twins - no AI writes CadQuery.
cadquery_worker.py	Out-of-process CadQuery worker with restricted builtins.
validation.py	Loads and validates the rendered mesh. Checks watertightness, attempts repair.
printability.py	The Printability Gate. Pass/warn/fail against the chosen printer and material.
orientation.py	Auto-orientation. Rotates the part to its most stable resting face.
hardening.py	Pre-slice mesh hardening via Manifold3D into a guaranteed 2-manifold.
licer.py	OrcaSlicer integration. Proves the result carries a real motion-bearing toolpath.
smart_mesh.py	Readiness synthesis. Folds the gate, mesh stats, and PrintProof3D into one verdict.
model_pull.py	In-app AI model download with progress reporting.
ollama_runtime.py	Managed AI engine lifecycle - download, start headless, teardown on exit.
paths.py	Dev/installed path seam. KIMCAD_INSTALL_ROOT switches all paths to install mode.
webapp.py	The local web layer. A dependency-free stdlib HTTP server over the same pipeline.
cli.py	The kimcad command. Wires all the pieces for terminal use.
design_store.py	The My Designs persistence layer. Local-first, best-effort, zip-slip safe.
design_registry.py	Per-design web state. Owns locks, LRU eviction, geometry-version guards.
printer_connector.py	The send-to-printer abstraction and the <code>ensure_sendable()</code> gate.
bambu_connector.py	Bambu Lab LAN connector (MQTT-over-TLS + FTPS).
octoprint_connector.py	OctoPrint REST connector.
moonraker_connector.py	Moonraker/Klipper REST connector.
prusalink_connector.py	PrusaLink REST connector.
duet_connector.py	Duet/RepRapFirmware connector.
marlin_connector.py	Marlin USB serial connector.
mcp_server.py	The printer MCP server (JSON-RPC over stdio).
model_advisor.py	Hardware-aware AI model recommendation.
config.py	Configuration loader. Layers <code>default.yaml</code> and <code>local.yaml</code> .

errors.py

Typed, user-facing error classes shared across CLI, pipeline, and web.

The Template Engine and Catalog

The template engine is the part of KimCad that makes live sliders possible. When your part matches one of the 86 built-in shapes, KimCad never calls the AI - it generates the OpenSCAD geometry by pure substitution from a typed template.

Every template has bounded, named parameters (width, height, wall thickness, etc.), an analytic bounding box that the Printability Gate asserts against, and a CadQuery twin for the editable STEP export. Re-rendering at a new parameter value takes well under a second.

The catalog spans eight theme groups:

- * Boxes and enclosures - project boxes, snap boxes, tubes, containers
- * Brackets and hardware - L-brackets, standoffs, plates, washers, fastener blanks
- * Organizers - drawer dividers, Gridfinity bins and baseplates, raceways
- * Hooks and clips - wall hooks, pegboard hooks, cable clips
- * Kim's decor world - frames, picture hangers, dishes and trays, candle holders, plant holders, planters, ornaments, gift boxes, stands, ledges, rails
- * Engineering hardware - Gridfinity, VESA mounts, thread-relief nut/bolt blanks
- * Outdoor and structural - slanted sign holders, Z-clip panel hangers
- * Utility - funnels, bar pulls, spool holders

Templates are honest about their tier. 'Benchmarked' means what-you-set-is-what-you-get, measured on the real pipeline. 'Baseline' means real, verified geometry with a real-world fit/load caveat to check before relying on it - for example, a thread-relief nut blank or a Gridfinity-compatible footprint.

Trust Boundaries

KimCad runs AI-generated code locally. The trust model is explicit:

- * AI-generated OpenSCAD is sanitized before it reaches the binary. File I/O operations (import, surface), CPU-bomb patterns (minkowski at high resolution), and any path outside the approved library are blocked - re-prompted, never stripped silently.
- * No AI ever writes CadQuery. The STEP path uses only KimCad's own trusted template scripts.
- * The CadQuery worker runs in a separate process with restricted builtins - no os, no open, no network - as defense-in-depth.
- * Credentials are read from environment variables only, never stored in config, never logged, and scrubbed from subprocess environments.
- * The web layer mints a fresh session token per boot and requires it on every state-changing request, blocking drive-by cross-origin POSTs.
- * The photo and sketch vision paths are local-only. The vision model is a dedicated local provider that cannot be re-routed to a cloud backend.

Testing Philosophy

The test suite (1,680 tests as of 0.9.2) is structured to prove the actual runtime contract, not just that the code runs:

- * The deterministic stages - validation, gate, orientation - are tested directly as pure functions.
- * The orchestration (pipeline.py) runs with a fake AI provider and a stub renderer against real trimesh geometry, so the retry loop, gate escape hatch, and confirm-before-slice rule are proven offline.
- * Live tests exercise the real OrcaSlicer binary, the real OpenSCAD binary, and the real Ollama model - the slice result is proven to carry a motion-bearing toolpath.
- * Every connector is tested against an adversarial runnable simulator with fault injection. A test that passes against a faithful simulator with Resend:/Error: faults is a test that earns its green.
- * The template engine is benchmarked: every family, re-rendered at new values, watertight at its declared envelope, byte-deterministic, with no model call.
- * The version is single-sourced (pyproject.toml) and enforced by a test that scans every surface - CLI, API, installer, Settings UI - for the canonical string.